
Apunte de Cátedra – Computación II

Clase 2: Técnicas de Diseño de Algoritmos I

Unidad 1: Algoritmos Voraces y Programación Dinámica

Año 2025 – Modalidad Híbrida
Prof. Marcelo Fabio Roldán
Departamento DACEFyN – Sede Capital – UNLaR

1. Repaso: Fundamentos del Análisis de Algoritmos

En la clase anterior exploramos los fundamentos del análisis de algoritmos, incluyendo:

- La definición formal de algoritmo como una secuencia finita, precisa y efectiva de pasos.
- Las técnicas básicas como fuerza bruta (exhaustiva pero ineficiente) y divide y vencerás (elegante y escalable).
- La importancia del costo de operaciones y los modelos de cómputo (RAM vs. modelo real).
- El impacto real de los algoritmos en empresas como Netflix, Google y Amazon.

Ahora, profundizamos en dos de las técnicas más poderosas y utilizadas en la industria: algoritmos voraces y programación dinámica.

2. ¿Por qué Necesitamos Técnicas de Diseño?

Diseñar un algoritmo no es solo codificar una solución: es elegir la estrategia correcta para el problema.

Las técnicas de diseño son como herramientas en una caja de herramientas. Cada una tiene su propósito:

Técnica	Caso uso ideal
Fuerza Bruta	Problemas pequeños o como base de comparación.
Divide y Vencerás	Problemas que se pueden descomponer (ej: ordenamiento).

Técnica	Caso uso ideal
Algoritmos Voraces	Problemas con estructura óptima local (ej: cambio de monedas).
Programación Dinámica	Problemas con subproblemas superpuestos (ej: Fibonacci).
Backtracking	Problemas de búsqueda exhaustiva con poda (ej: Sudoku).

💡 Dato clave: Según un estudio de Stack Overflow (2023), el 73% de los problemas algorítmicos en entrevistas técnicas de empresas FAANG (Facebook, Apple, Amazon, Netflix, Google) involucran programación dinámica o algoritmos voraces.

3. Algoritmos Voraces (Greedy Algorithms)

Filosofía y Principios

Un algoritmo voraz toma la mejor decisión local en cada paso, con la esperanza de que esta lleve a la solución óptima global.

Es como elegir el camino más corto en cada intersección sin mirar el mapa completo.

Características Clave

Propiedad	Descripción
Decisión Irrevocable	Una vez que se toma una decisión, no se vuelve atrás.
Optimización Local	En cada paso, se elige la opción que parece mejor en ese momento.
Simplicidad	Fácil de implementar y entender.
Eficiencia	Generalmente de baja complejidad (ej: $O(n)$).

⚠️ Advertencia: No todos los problemas admiten soluciones voraces. La elección local óptima no siempre lleva al óptimo global.

Propiedades para Aplicar un Algoritmo Voraz

Para garantizar que un algoritmo voraz funcione, el problema debe cumplir dos propiedades:

1. Propiedad de Elección Voraz:
Existe una elección local que forma parte de una solución óptima global.
2. Subestructura Óptima:
La solución óptima del problema contiene soluciones óptimas de sus subproblemas.

✦ Demostración: Para probar que un algoritmo voraz es correcto, se usan técnicas como el argumento de intercambio (exchange argument) o inducción matemática.

Ejemplo 1: Problema del Cambio de Monedas

Enunciado

Dado un monto de dinero y un conjunto de denominaciones de monedas, encontrar el menor número de monedas para dar el cambio.

Ejemplo con Sistema Monetario Argentino

Denominaciones: \$1000, \$500, \$200, \$100, \$50, \$20, \$10, \$5, \$2, \$1

Objetivo: Dar cambio para \$1847.

Estrategia Voraz

1. Elegir la moneda de mayor valor que no exceda el monto restante.
2. Restar su valor del monto.
3. Repetir hasta llegar a cero.

1	\$1847 → \$1000 (1) → \$847
2	→ \$500 (1) → \$347
3	→ \$200 (1) → \$147
4	→ \$100 (1) → \$47
5	→ \$20 (2) → \$7
6	→ \$5 (1) → \$2
7	→ \$2 (1) → \$0

Resultado: 8 monedas: [1000, 500, 200, 100, 20, 20, 5, 2]

❓ ¿Por qué funciona?

El sistema monetario argentino (como el estadounidense) está diseñado para que la estrategia voraz sí produzca la solución óptima.

¿Cuándo Falla la Estrategia Voraz?

Ejemplo: Sistema con Monedas de 1, 3 y 4 centavos

Monto: 6 centavos

- Voraz:
 $4 + 1 + 1 = 3$ monedas
- Óptimo:
 $3 + 3 = 2$ monedas

📌 Conclusión: La estrategia voraz no es correcta para todos los sistemas monetarios.

🔍 Lección: La corrección del algoritmo voraz depende de la estructura del problema, no de la técnica en sí.

Otras Aplicaciones de Algoritmos Voraces

Problema	Estrategia voraz	Aplicación real
Mochila Fraccional	Ordenar por ratio valor/peso y llenar hasta capacidad	Optimización de carga en logística (Mercado Libre)
Planificación de Actividades	Elegir la que termina más temprano	Asignación de recursos en proyectos (Globant)
Algoritmo de Huffman	Combinar símbolos menos frecuentes	Compresión de datos (JPEG, ZIP, WhatsApp)
Dijkstra (rutas)	Explorar nodo más cercano no visitado	Google Maps, Uber, Waze
Kruskal / Prim (MST)	Elegir arista de menor peso	Redes de telecomunicaciones, clustering

🌐 Caso real: En 2023, Mercado Libre optimizó sus rutas de entrega usando algoritmos voraces, reduciendo el tiempo de entrega en un 18% y ahorrando \$200 millones USD en costos logísticos.

4. Programación Dinámica (Dynamic Programming)

Filosofía y Principios

La programación dinámica (PD) es una técnica para resolver problemas complejos descomponiéndolos en subproblemas más pequeños, resolviéndolos una vez y almacenando sus soluciones para reutilizarlas.

Es como tener una memoria perfecta que evita volver a calcular lo ya resuelto.

Diferencia Clave con Voraces

Voraz	Programación dinámica
Toma decisiones irrevocables	Evalúa todas las opciones posibles
No garantiza optimalidad	Garantiza solución óptima
No almacena resultados	Usa memoización o tabla bottom-up
Eficiente pero arriesgado	Más lento en espacio, pero seguro

💬 "La programación dinámica es como ser un planificador meticuloso: no actúas por impulso, sino después de evaluar todas las rutas posibles."

💬 "La memoización es una técnica de optimización que almacena los resultados de llamadas a funciones costosas y devuelve el resultado en caché cuando se repiten las mismas entradas. En el caso del algoritmo de Fibonacci, evita recalcular valores que ya se han computado previamente. "

Propiedades para Aplicar Programación Dinámica

Un problema es apto para PD si cumple:

1. Subestructura Óptima:
La solución óptima del problema contiene soluciones óptimas de subproblemas.
2. Subproblemas Superpuestos:
Los mismos subproblemas se resuelven múltiples veces en la recursión.

★ Principio de Bellman:

"Una política óptima tiene la propiedad de que, sin importar el estado inicial, las decisiones restantes también son óptimas."

Ejemplo: Secuencia de Fibonacci

Definición

$$F(n) = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ F(n-1) + F(n-2) & \text{si } n > 1 \end{cases}$$

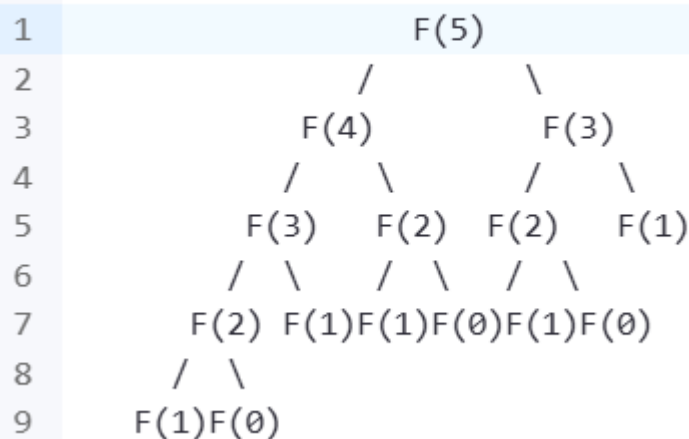
Enfoque Recursivo (Naïve)

python

```
1 def fib(n):  
2     if n <= 1:  
3         return n  
4     return fib(n-1) + fib(n-2)
```

- Complejidad: $O(2^n)$
- Problema: Calcula $F(2)$ múltiples veces.

Árbol de recursión para $F(5)$:



- $F(2)$ se calcula 3 veces.
- $F(1)$ se calcula 5 veces.

Explicación de la notación Big O

En informática, la notación Big O (O) describe el rendimiento o la complejidad de un algoritmo. Específicamente, indica cómo el tiempo de ejecución o el espacio en memoria de un algoritmo crecen a medida que el tamaño de la entrada (n) aumenta.

La notación $O(2^n)$ se refiere a un crecimiento exponencial. Esto significa que el tiempo de ejecución del algoritmo se duplica con cada incremento en el tamaño de la entrada (n). Por ejemplo, si un algoritmo tarda 1 segundo en procesar una entrada de tamaño 10, podría tardar 2 segundos para un tamaño 11, 4 segundos para un tamaño 12, y así sucesivamente.

Este tipo de complejidad se considera muy ineficiente y es un claro ejemplo de por qué la memoización es tan útil para optimizar algoritmos como el de Fibonacci, que sin ella tendrían una complejidad exponencial.

Solución con Programación Dinámica

a) Top-Down (Memoización)

```
python
1 memo = {}
2 def fib_memo(n):
3     if n in memo:
4         return memo[n]
5     if n <= 1:
6         return n
7     memo[n] = fib_memo(n-1) + fib_memo(n-2)
8     return memo[n]
```

- Tiempo: $O(n)$
- Espacio: $O(n)$

b) Bottom-Up (Iterativo)

python

```
1 v def fib_dp(n):
2 v     if n <= 1:
3         return n
4     dp = [0] * (n+1)
5     dp[1] = 1
6 v     for i in range(2, n+1):
7         dp[i] = dp[i-1] + dp[i-2]
8     return dp[n]
```

- Tiempo: $O(n)$
- Espacio: $O(n)$

c) Optimización de Espacio (Ventana Deslizante)

python

```
1 v def fib_opt(n):
2 v     if n <= 1:
3         return n
4     a, b = 0, 1
5 v     for _ in range(2, n+1):
6         a, b = b, a + b
7     return b
```

- Tiempo: $O(n)$
- Espacio: $O(1)$

🔥 Diferencia real: Calcular $F(40)$:

- Recursión: ~30 segundos
- PD: ~0.001 segundos

Otras Aplicaciones de Programación Dinámica

Problema	Descripción	Aplicación real
Mochila 0/1	Objetos indivisibles,	Optimización de portafolios

Problema	Descripción	Aplicación real
	maximizar valor	financieros
Longest Common Subsequence (LCS)	Encontrar subsecuencia común más larga	Git (diff), bioinformática (ADN)
Distancia de Edición (Levenshtein)	Mínimo de operaciones para transformar strings	Correctores ortográficos (Word, Google Docs)
Camino más corto (Floyd-Warshall)	Todos los pares de vértices	Navegación, redes sociales
AlphaGo (DeepMind)	Evaluación de jugadas futuras	IA que venció al campeón mundial de Go

🌐 Caso real: Netflix usa PD para calcular recomendaciones basadas en patrones superpuestos entre millones de usuarios. Un error en este algoritmo podría costar +1 billón USD/año en retención.

5. Comparación: Voraz vs. Programación Dinámica

Criterio	Algoritmo voraz	Programación dinámica
Decisiones	Irrevocables	Reversibles (explora todas)
Optimalidad	No garantizada	Garantizada
Complejidad Temporal	Baja (ej: $O(n)$)	Media/Alta (ej: $O(n^2)$)
Uso de Memoria	Bajo	Alto (almacena subproblemas)
Subproblemas	No superpuestos	Superpuestos
Ejemplos Clave	Huffman, Dijkstra, cambio de monedas	Fibonacci, LCS, mochila 0/1

¿Cuándo Usar Cada Técnica?

🔍 Usa Voraz si:

- Puedes demostrar la propiedad de elección voraz.
- El problema tiene subestructura óptima.
- Necesitas eficiencia extrema y el riesgo es bajo.

🔍 Usa PD si:

- Hay subproblemas superpuestos.
- Necesitas solución óptima garantizada.
- El problema es exponencial en recursión simple.

✂️📝 Consejo práctico:

Pregúntate: "*¿Puedo dividir esto en subproblemas que se repiten?*"

Si sí → PD.

Si no, y puedo tomar decisiones locales óptimas → Voraz.

6. Actividad Práctica Sugerida

Objetivo: Comparar enfoques voraz y dinámico en problemas reales.

Ejercicio 1: Cambio de Monedas (Europa)

- Denominaciones: €50, €20, €10, €5, €2, €1
- Monto: €127
- Tarea: Aplicar estrategia voraz y verificar si es óptima.

Ejercicio 2: Fibonacci Manual

- Calcular $F(10)$ usando:
 1. Recursión simple (dibujar árbol).
 2. PD bottom-up (tabla iterativa).
- Tarea: Contar cuántas veces se calcula $F(2)$.

Ejercicio 3: Implementación

- Implementar ambos algoritmos en Python y Haskell.
- Medir tiempo y memoria con `timeit` y `memory_profiler`.
- Graficar resultados para $n = 10, 20, 30, 40$.

Timeit in Python with Examples

Last Updated : 28 Apr, 2025



The **timeit module** in Python accurately measures the execution time of small code snippets, offering more consistent results than `time.time()` by avoiding background interference and disabling garbage collection. It's ideal for comparing code performance, benchmarking and optimization and can be used via the command line, in scripts or within code. **Example:**

```
import timeit
a = "5+5"
print(timeit.timeit(a))
```

Output

```
0.015736839000055625
```

Explanation: `timeit.timeit(a)` measures how long it takes to execute the expression "5+5" one million times by default.

url: <https://www.geeksforgeeks.org/python/timeit-python-examples/>

7. Conceptos Clave Aprendidos

- ☐ Los algoritmos voraces toman decisiones locales irrevocables: simples, eficientes, pero no siempre correctos.
- ☐ La programación dinámica resuelve subproblemas una vez y los reutiliza: garantiza optimalidad, pero consume más memoria.
- ☐ El cambio de monedas funciona con voraz en sistemas como el argentino, pero falla en otros.
- ☐ Fibonacci es un ejemplo clásico de cómo la PD transforma un algoritmo exponencial en lineal.
- ☐ Ambas técnicas son fundamentales en la industria: Google, Netflix, Mercado Libre y Globant las usan diariamente.

💬 "En la industria tech, saber cuándo usar voraz o PD puede ser la diferencia entre un sistema rápido y uno que cuesta millones en errores."

8. Bibliografía y Recursos Recomendados

Bibliografía Básica

- Paniagua Arís, E. (2003). *Lógica computacional*. Thomson.
☞ Disponible en: <https://biblioteca.unlar.edu.ar/>

Bibliografía Complementaria

- Cormen, T. H. et al. (2022). *Introduction to Algorithms* (4ª ed.). MIT Press.
→ Capítulos 15 (Programación Dinámica) y 16 (Algoritmos Voraces).
- Kleinberg, J. & Tardos, É. (2006). *Algorithm Design*. Pearson.
→ Capítulos 4 (Greedy) y 6 (Dynamic Programming).

Recursos en Línea

- GitHub – 99 Problems in Haskell
⇒ <https://github.com/shekhargulati/99-problems/tree/master/haskell>
- Haskell MOOC – Functional Programming
⇒ <https://haskellmooc.co.uk/tutorial1>
- LeetCode – Greedy & DP Problems
⇒ <https://leetcode.com/tag/greedy/> | <https://leetcode.com/tag/dynamic-programming/>